SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 5 **Total Marks: 100**

ANNEXURE A **Coding Challenge**

Code of Conduct:

I J KARTHIK / 192472136 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *J KARTHIK*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor.

Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Question 1: Library Book Management using Classes & Encapsulation

Source Code: LibraryManagement.java

```
import java.util.Scanner;

class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private boolean issued;

    public Book(int bookId, String title, String author, double price) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.price = price;
        this.issued = false;
    }

    public int getBookId() { return bookId; }
    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public double getPrice() { return price; }
    public boolean isIssued() { return issued; }

    public void setTitle(String title) { this.title = title; }
    public void setAuthor(String author) { this.author = author; }
    public void setPrice(double price) { this.price = price; }

    public boolean issueBook() {
        if (issued) {
            System.out.println("Book \"" + title + "\" is already issued.");
            return false;
        }
        issued = true;
        System.out.println("Book \"" + title + "\" issued successfully.");
        return true;
    }

    public boolean returnBook() {
        if (!issued) {
            System.out.println("Book \"" + title + "\" was not issued.");
            return false;
        }
        issued = false;
        System.out.println("Book \"" + title + "\" returned successfully.");
        return true;
    }

    public void display() {
        System.out.printf("ID: %-4d Title: %-20s Author: %-15s Price: %-8.2f Status: %s%n",
                bookId, title, author, price, issued ? "ISSUED" : "AVAILABLE");
    }
}

public class LibraryManagement {
    private static Book[] books = new Book[100];
    private static int count = 0;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n--- Library Book Management ---");
            System.out.println("1. Add Book");
            System.out.println("2. Issue Book");
            System.out.println("3. Return Book");
            System.out.println("4. Search Book");
            System.out.println("5. Display All Books");
```

```
            System.out.println("6. Exit");
            System.out.print("Enter choice: ");
            choice = Integer.parseInt(sc.nextLine().trim());

            switch (choice) {
                case 1: addBook(sc); break;
                case 2: issueBook(sc); break;
                case 3: returnBook(sc); break;
                case 4: searchBook(sc); break;
                case 5: displayAll(); break;
                case 6: System.out.println("Exiting..."); break;
                default: System.out.println("Invalid choice!");
            }
        } while (choice != 6);
        sc.close();
    }

    private static void addBook(Scanner sc) {
        System.out.print("Enter Book ID: ");
        int id = Integer.parseInt(sc.nextLine().trim());
        System.out.print("Enter Title: ");
        String title = sc.nextLine();
        System.out.print("Enter Author: ");
        String author = sc.nextLine();
        System.out.print("Enter Price: ");
        double price = Double.parseDouble(sc.nextLine().trim());
        books[count++] = new Book(id, title, author, price);
        System.out.println("Book added successfully.");
    }

    private static Book findBook(int id) {
        for (int i = 0; i < count; i++) {
            if (books[i].getBookId() == id) return books[i];
        }
        return null;
    }

    private static void issueBook(Scanner sc) {
        System.out.print("Enter Book ID to issue: ");
        int id = Integer.parseInt(sc.nextLine().trim());
        Book b = findBook(id);
        if (b == null) System.out.println("Book not found!");
        else b.issueBook();
    }

    private static void returnBook(Scanner sc) {
        System.out.print("Enter Book ID to return: ");
        int id = Integer.parseInt(sc.nextLine().trim());
        Book b = findBook(id);
        if (b == null) System.out.println("Book not found!");
        else b.returnBook();
    }

    private static void searchBook(Scanner sc) {
        System.out.print("Enter Book ID or Title to search: ");
        String key = sc.nextLine().trim();
        boolean found = false;
        for (int i = 0; i < count; i++) {
            if (String.valueOf(books[i].getBookId()).equals(key) ||
                books[i].getTitle().equalsIgnoreCase(key)) {
                books[i].display();
                found = true;
            }
        }
        if (!found) System.out.println("No matching book found.");
    }

    private static void displayAll() {
        if (count == 0) { System.out.println("No books available."); return; }
        for (int i = 0; i < count; i++) books[i].display();
    }
}
```

Program Output

```
--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter choice: ID: 101  Title: Java Programming     Author: James Gosling   Price: 650.00   Status:
AVAILABLE
ID: 102  Title: Data Structures      Author: Robert Lafore   Price: 550.00   Status: AVAILABLE
ID: 103  Title: Operating Systems    Author: Galvin          Price: 700.00   Status: AVAILABLE

--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter choice: Enter Book ID to issue: Book "Java Programming" issued successfully.

--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter choice: Enter Book ID to issue: Book "Java Programming" is already issued.

--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter choice: Enter Book ID or Title to search: ID: 101  Title: Java Programming     Author: James
Gosling   Price: 650.00   Status: ISSUED

--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter choice: Enter Book ID or Title to search: No matching book found.

--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter choice: Enter Book ID to return: Book "Java Programming" returned successfully.

--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
4. Search Book
5. Display All Books
6. Exit
Enter choice: ID: 101  Title: Java Programming     Author: James Gosling   Price: 650.00   Status:
AVAILABLE
ID: 102  Title: Data Structures      Author: Robert Lafore   Price: 550.00   Status: AVAILABLE
ID: 103  Title: Operating Systems    Author: Galvin          Price: 700.00   Status: AVAILABLE

--- Library Book Management ---
1. Add Book
2. Issue Book
3. Return Book
```

```
4. Search Book
5. Display All Books
6. Exit
Enter choice: Exiting...
```

Question 2: Employee Payroll System using Inheritance

Source Code: PayrollSystem.java

```
class Employee {
    protected int empId;
    protected String name;
    protected double basicPay;

    public Employee(int empId, String name, double basicPay) {
        this.empId = empId;
        this.name = name;
        this.basicPay = basicPay;
    }

    public double calculateSalary() {
        return basicPay;
    }

    public void display() {
        System.out.printf("ID: %-5d Name: %-12s Type: %-20s Salary: Rs.%.2f%n",
                empId, name, this.getClass().getSimpleName(), calculateSalary());
    }
}

class PermanentEmployee extends Employee {
    private double bonus;
    private double pf;

    public PermanentEmployee(int empId, String name, double basicPay, double bonus, double pf) {
        super(empId, name, basicPay);
        this.bonus = bonus;
        this.pf = pf;
    }

    @Override
    public double calculateSalary() {
        return basicPay + bonus - pf;
    }
}

class ContractEmployee extends Employee {
    private int hoursWorked;
    private double ratePerHour;

    public ContractEmployee(int empId, String name, int hoursWorked, double ratePerHour) {
        super(empId, name, 0);
        this.hoursWorked = hoursWorked;
        this.ratePerHour = ratePerHour;
    }

    @Override
    public double calculateSalary() {
        return hoursWorked * ratePerHour;
    }
}

public class PayrollSystem {
    public static void main(String[] args) {
        Employee[] employees = new Employee[4];
        employees[0] = new PermanentEmployee(1, "Arun", 30000, 5000, 2000);
        employees[1] = new PermanentEmployee(2, "Divya", 40000, 6000, 2500);
        employees[2] = new ContractEmployee(3, "Karthik", 160, 250);
        employees[3] = new ContractEmployee(4, "Meena", 140, 300);

        System.out.println("---- Employee Payroll Report ----");
        double totalPayout = 0;
        for (Employee e : employees) {
```

```
            e.display();
            totalPayout += e.calculateSalary();
        }
        System.out.printf("%nTotal Payout for all employees: Rs.%.2f%n", totalPayout);

        System.out.println("\n---- Test Cases ----");
        Employee test1 = new PermanentEmployee(101, "TestPerm", 25000, 3000, 1500);
        System.out.printf("Test1 (Permanent) Expected=26500.00 Actual=%.2f%n", test1.calculateSalary());

        Employee test2 = new ContractEmployee(102, "TestContract", 100, 200);
        System.out.printf("Test2 (Contract) Expected=20000.00 Actual=%.2f%n", test2.calculateSalary());
    }
}
```

Program Output

```
---- Employee Payroll Report ----
ID: 1     Name: Arun         Type: PermanentEmployee    Salary: Rs.33000.00
ID: 2     Name: Divya        Type: PermanentEmployee    Salary: Rs.43500.00
ID: 3     Name: Karthik      Type: ContractEmployee     Salary: Rs.40000.00
ID: 4     Name: Meena        Type: ContractEmployee     Salary: Rs.42000.00

Total Payout for all employees: Rs.158500.00

---- Test Cases ----
Test1 (Permanent) Expected=26500.00 Actual=26500.00
Test2 (Contract) Expected=20000.00 Actual=20000.00
```

Question 3: Vehicle Rental System using Runtime Polymorphism

Source Code: VehicleRental.java

```
class Vehicle {
    protected String vehicleNo;
    protected String model;

    public Vehicle(String vehicleNo, String model) {
        this.vehicleNo = vehicleNo;
        this.model = model;
    }

    public double calculateRent(int days) {
        return 0;
    }

    public void generateBill(int days) {
        System.out.printf("Vehicle No: %-10s Model: %-10s Type: %-6s Days: %-3d Rent: Rs.%.2f%n",
                vehicleNo, model, this.getClass().getSimpleName(), days, calculateRent(days));
    }
}

class Car extends Vehicle {
    public Car(String vehicleNo, String model) { super(vehicleNo, model); }
    @Override
    public double calculateRent(int days) { return days * 1500.0; }
}

class Bike extends Vehicle {
    public Bike(String vehicleNo, String model) { super(vehicleNo, model); }
    @Override
    public double calculateRent(int days) { return days * 500.0; }
}

class Bus extends Vehicle {
    public Bus(String vehicleNo, String model) { super(vehicleNo, model); }
    @Override
    public double calculateRent(int days) { return days * 4000.0; }
}

public class VehicleRental {
    public static void main(String[] args) {
```

```
        Vehicle[] vehicles = new Vehicle[3];
        vehicles[0] = new Car("TN01AB1234", "Swift");
        vehicles[1] = new Bike("TN02CD5678", "Activa");
        vehicles[2] = new Bus("TN03EF9012", "Volvo");

        int[] rentalDays = {3, 5, 2};

        System.out.println("---- Vehicle Rental Bills ----");
        double grandTotal = 0;
        for (int i = 0; i < vehicles.length; i++) {
            vehicles[i].generateBill(rentalDays[i]);
            grandTotal += vehicles[i].calculateRent(rentalDays[i]);
        }
        System.out.printf("%nGrand Total Rent Collected: Rs.%.2f%n", grandTotal);

        System.out.println("\n---- Test Cases ----");
        Vehicle testCar = new Car("TEST01", "TestCar");
        System.out.printf("Car 4 days -> Expected=6000.00 Actual=%.2f%n", testCar.calculateRent(4));

        Vehicle testBike = new Bike("TEST02", "TestBike");
        System.out.printf("Bike 4 days -> Expected=2000.00 Actual=%.2f%n", testBike.calculateRent(4));

        Vehicle testBus = new Bus("TEST03", "TestBus");
        System.out.printf("Bus 4 days -> Expected=16000.00 Actual=%.2f%n", testBus.calculateRent(4));
    }
}
```

Program Output

```
---- Vehicle Rental Bills ----
Vehicle No: TN01AB1234 Model: Swift      Type: Car    Days: 3   Rent: Rs.4500.00
Vehicle No: TN02CD5678 Model: Activa     Type: Bike   Days: 5   Rent: Rs.2500.00
Vehicle No: TN03EF9012 Model: Volvo      Type: Bus    Days: 2   Rent: Rs.8000.00

Grand Total Rent Collected: Rs.15000.00

---- Test Cases ----
Car 4 days -> Expected=6000.00 Actual=6000.00
Bike 4 days -> Expected=2000.00 Actual=2000.00
Bus 4 days -> Expected=16000.00 Actual=16000.00
```

Question 4: Bank Account System using Data Hiding

Source Code: BankAccountSystem.java

```
class BankAccount {
    private String accountNumber;
    private String holderName;
    private double balance;

    public BankAccount(String accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = (balance >= 0) ? balance : 0;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getHolderName() { return holderName; }

    public double getBalance() {
        return balance;
    }

    public boolean deposit(double amount) {
        if (amount <= 0) {
            System.out.println("Deposit failed: Amount must be positive.");
            return false;
        }
        balance += amount;
        System.out.printf("Deposited Rs.%.2f successfully. New Balance: Rs.%.2f%n", amount, balance);
        return true;
```

```
    }

    public boolean withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Withdrawal failed: Amount must be positive.");
            return false;
        }
        if (amount > balance) {
            System.out.println("Withdrawal failed: Insufficient balance.");
            return false;
        }
        balance -= amount;
        System.out.printf("Withdrew Rs.%.2f successfully. New Balance: Rs.%.2f%n", amount, balance);
        return true;
    }

    public void balanceInquiry() {
        System.out.printf("Account: %s (%s) -> Balance: Rs.%.2f%n", accountNumber, holderName, balance);
    }
}

public class BankAccountSystem {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("AC1001", "Karthik", 5000.0);

        System.out.println("---- Bank Account Transactions ----");
        acc.balanceInquiry();

        System.out.println("\nTest 1: Valid deposit");
        acc.deposit(2000);

        System.out.println("\nTest 2: Invalid deposit (negative amount)");
        acc.deposit(-500);

        System.out.println("\nTest 3: Valid withdrawal");
        acc.withdraw(3000);

        System.out.println("\nTest 4: Invalid withdrawal (exceeds balance)");
        acc.withdraw(100000);

        System.out.println("\nTest 5: Invalid withdrawal (negative amount)");
        acc.withdraw(-50);

        System.out.println("\nFinal Balance Inquiry:");
        acc.balanceInquiry();
    }
}
```

Program Output

```
---- Bank Account Transactions ----
Account: AC1001 (Karthik) -> Balance: Rs.5000.00

Test 1: Valid deposit
Deposited Rs.2000.00 successfully. New Balance: Rs.7000.00

Test 2: Invalid deposit (negative amount)
Deposit failed: Amount must be positive.

Test 3: Valid withdrawal
Withdrew Rs.3000.00 successfully. New Balance: Rs.4000.00

Test 4: Invalid withdrawal (exceeds balance)
Withdrawal failed: Insufficient balance.

Test 5: Invalid withdrawal (negative amount)
Withdrawal failed: Amount must be positive.

Final Balance Inquiry:
Account: AC1001 (Karthik) -> Balance: Rs.4000.00
```

Question 5: University Admission System using Method Overloading

Source Code: AdmissionSystem.java

```
class Admission {
    // Undergraduate students
    public double calculateFee(String course, int year) {
        double baseFee = 50000;
        double fee = baseFee + (year * 2000);
        System.out.printf("[UG] Course: %-10s Year: %d -> Fee: Rs.%.2f%n", course, year, fee);
        return fee;
    }

    // Postgraduate students
    public double calculateFee(String course, int year, boolean hostel) {
        double baseFee = 75000;
        double fee = baseFee + (year * 3000) + (hostel ? 20000 : 0);
        System.out.printf("[PG] Course: %-10s Year: %d Hostel: %-5s -> Fee: Rs.%.2f%n", course, year,
hostel, fee);
        return fee;
    }

    // Scholarship students
    public double calculateFee(String course, double scholarshipPercent) {
        double baseFee = 60000;
        double discount = baseFee * (scholarshipPercent / 100);
        double fee = baseFee - discount;
        System.out.printf("[Scholarship] Course: %-10s Scholarship: %.1f%% -> Fee: Rs.%.2f%n", course,
scholarshipPercent, fee);
        return fee;
    }
}

public class AdmissionSystem {
    public static void main(String[] args) {
        Admission admission = new Admission();

        System.out.println("---- University Admission Fee Structure ----");
        admission.calculateFee("B.Tech", 1);
        admission.calculateFee("M.Tech", 1, true);
        admission.calculateFee("B.Sc", 25.0);

        System.out.println("\n---- Comparing Overloaded Executions ----");
        double ugFee = admission.calculateFee("BCA", 2);
        double pgFee = admission.calculateFee("MBA", 2, false);
        double schFee = admission.calculateFee("B.Com", 50.0);

        System.out.println("\n---- Test Cases ----");
        System.out.printf("Test UG (course, year=3) Expected=56000.00 Actual=%.2f%n",
admission.calculateFee("B.Tech", 3));
        System.out.printf("Test PG (course, year=2, hostel=true) Expected=101000.00 Actual=%.2f%n",
admission.calculateFee("M.Tech", 2, true));
        System.out.printf("Test PG (course, year=2, hostel=false) Expected=81000.00 Actual=%.2f%n",
admission.calculateFee("M.Tech", 2, false));
        System.out.printf("Test Scholarship (course, 100%%) Expected=0.00 Actual=%.2f%n",
admission.calculateFee("B.Sc", 100.0));
    }
}
```

Program Output

```
---- University Admission Fee Structure ----
[UG] Course: B.Tech     Year: 1 -> Fee: Rs.52000.00
[PG] Course: M.Tech     Year: 1 Hostel: true  -> Fee: Rs.98000.00
[Scholarship] Course: B.Sc       Scholarship: 25.0% -> Fee: Rs.45000.00

---- Comparing Overloaded Executions ----
[UG] Course: BCA        Year: 2 -> Fee: Rs.54000.00
[PG] Course: MBA        Year: 2 Hostel: false -> Fee: Rs.81000.00
[Scholarship] Course: B.Com      Scholarship: 50.0% -> Fee: Rs.30000.00

---- Test Cases ----
[UG] Course: B.Tech     Year: 3 -> Fee: Rs.56000.00
Test UG (course, year=3) Expected=56000.00 Actual=56000.00
```

```
[PG] Course: M.Tech     Year: 2 Hostel: true  -> Fee: Rs.101000.00
Test PG (course, year=2, hostel=true) Expected=101000.00 Actual=101000.00
[PG] Course: M.Tech     Year: 2 Hostel: false -> Fee: Rs.81000.00
Test PG (course, year=2, hostel=false) Expected=81000.00 Actual=81000.00
[Scholarship] Course: B.Sc       Scholarship: 100.0% -> Fee: Rs.0.00
Test Scholarship (course, 100%) Expected=0.00 Actual=0.00
```

Question 6: Shape Area Calculator using Abstract Class & Polymorphism

Source Code: ShapeAreaCalculator.java

```
abstract class Shape {
    public abstract double area();
    public void display() {
        System.out.printf("%-10s -> Area: %.2f%n", this.getClass().getSimpleName(), area());
    }
}

class Circle extends Shape {
    private double radius;
    public Circle(double radius) { this.radius = radius; }
    @Override
    public double area() { return Math.PI * radius * radius; }
}

class Rectangle extends Shape {
    private double length, width;
    public Rectangle(double length, double width) { this.length = length; this.width = width; }
    @Override
    public double area() { return length * width; }
}

class Triangle extends Shape {
    private double base, height;
    public Triangle(double base, double height) { this.base = base; this.height = height; }
    @Override
    public double area() { return 0.5 * base * height; }
}

public class ShapeAreaCalculator {
    public static void main(String[] args) {
        Shape[] shapes = new Shape[3];
        shapes[0] = new Circle(5);
        shapes[1] = new Rectangle(4, 6);
        shapes[2] = new Triangle(8, 5);

        System.out.println("---- Shape Area Calculator (Polymorphic Behavior) ----");
        for (Shape s : shapes) {
            s.display();
        }

        System.out.println("\n---- Test Cases ----");
        Shape c = new Circle(7);
        System.out.printf("Circle r=7 Expected=153.94 Actual=%.2f%n", c.area());

        Shape r = new Rectangle(10, 5);
        System.out.printf("Rectangle 10x5 Expected=50.00 Actual=%.2f%n", r.area());

        Shape t = new Triangle(10, 4);
        System.out.printf("Triangle base=10 height=4 Expected=20.00 Actual=%.2f%n", t.area());
    }
}
```

Program Output

```
---- Shape Area Calculator (Polymorphic Behavior) ----
Circle     -> Area: 78.54
Rectangle  -> Area: 24.00
Triangle   -> Area: 20.00

---- Test Cases ----
Circle r=7 Expected=153.94 Actual=153.94
```

```
Rectangle 10x5 Expected=50.00 Actual=50.00
Triangle base=10 height=4 Expected=20.00 Actual=20.00
```

Question 7: Hospital Management System using Interfaces

Source Code: HospitalManagement.java

```
import java.util.Scanner;

interface MedicalRecord {

    void addRecord();

    void displayRecord();
}

class Patient implements MedicalRecord {

    int patientId;
    String patientName;
    String disease;

    Scanner sc = new Scanner(System.in);

    public void addRecord() {

        System.out.print("Enter Patient ID: ");
        patientId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Patient Name: ");
        patientName = sc.nextLine();

        System.out.print("Enter Disease: ");
        disease = sc.nextLine();
    }

    public void displayRecord() {

        System.out.println("\n----- Patient Record -----");
        System.out.println("Patient ID   : " + patientId);
        System.out.println("Patient Name : " + patientName);
        System.out.println("Disease      : " + disease);
    }
}

class Doctor implements MedicalRecord {

    int doctorId;
    String doctorName;
    String specialization;

    Scanner sc = new Scanner(System.in);

    public void addRecord() {

        System.out.print("\nEnter Doctor ID: ");
        doctorId = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Doctor Name: ");
        doctorName = sc.nextLine();

        System.out.print("Enter Specialization: ");
        specialization = sc.nextLine();
    }

    public void displayRecord() {

        System.out.println("\n----- Doctor Record -----");
        System.out.println("Doctor ID        : " + doctorId);
        System.out.println("Doctor Name      : " + doctorName);
        System.out.println("Specialization   : " + specialization);
    }
```

```
}

public class Main {

    public static void main(String[] args) {

        Patient p = new Patient();
        Doctor d = new Doctor();

        p.addRecord();
        d.addRecord();

        p.displayRecord();
        d.displayRecord();
    }
}
```

Program Output

```
Enter Patient ID: 101
Enter Patient Name: Karthik
Enter Disease: Fever

Enter Doctor ID: 201
Enter Doctor Name: Dr. Kumar
Enter Specialization: Cardiology

----- Patient Record -----
Patient ID   : 101
Patient Name : Karthik
Disease      : Fever

----- Doctor Record -----
Doctor ID        : 201
Doctor Name      : Dr. Kumar
Specialization   : Cardiology
```

Question 8: Online Shopping Order System using Packages

Source Code: Product.java

```
package shopping.product;

public class Product {
    private String productId;
    private String name;
    private double price;
    private int quantity;

    public Product(String productId, String name, double price, int quantity) {
        this.productId = productId;
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    public String getProductId() { return productId; }
    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getQuantity() { return quantity; }

    public double getSubTotal() {
        return price * quantity;
    }

    public void display() {
        System.out.printf("Product: %-12s Price: Rs.%-8.2f Qty: %-3d SubTotal: Rs.%.2f%n",
                name, price, quantity, getSubTotal());
    }
}
```

Source Code: Order.java

```
package shopping.order;
```

```
import shopping.product.Product;
import java.util.ArrayList;
import java.util.List;

public class Order {
    private String orderId;
    private List<Product> products = new ArrayList<>();

    public Order(String orderId) {
        this.orderId = orderId;
    }

    public void addProduct(Product p) {
        products.add(p);
        System.out.println("Added to order " + orderId + ": " + p.getName());
    }

    public double calculateTotal() {
        double total = 0;
        for (Product p : products) total += p.getSubTotal();
        return total;
    }

    public void generateBill() {
        System.out.println("\n---- Bill for Order: " + orderId + " ----");
        for (Product p : products) p.display();
        System.out.printf("Total Amount: Rs.%.2f%n", calculateTotal());
    }
}
```

Source Code: OnlineShopping.java

```
import shopping.product.Product;
import shopping.order.Order;

public class OnlineShopping {
    public static void main(String[] args) {
        System.out.println("---- Online Shopping Order System ----");
        Order order1 = new Order("ORD1001");
        order1.addProduct(new Product("P001", "Laptop", 55000.0, 1));
        order1.addProduct(new Product("P002", "Mouse", 500.0, 2));
        order1.addProduct(new Product("P003", "Keyboard", 1200.0, 1));
        order1.generateBill();

        System.out.println("\n---- Test Cases ----");
        Order testOrder = new Order("TEST01");
        testOrder.addProduct(new Product("T001", "TestItem", 100.0, 3));
        double expected = 300.0;
        System.out.printf("Expected Total=%.2f Actual Total=%.2f%n", expected,
testOrder.calculateTotal());
        testOrder.generateBill();
    }
}
```

Program Output

```
---- Online Shopping Order System ----
Added to order ORD1001: Laptop
Added to order ORD1001: Mouse
Added to order ORD1001: Keyboard

---- Bill for Order: ORD1001 ----
Product: Laptop       Price: Rs.55000.00 Qty: 1   SubTotal: Rs.55000.00
Product: Mouse        Price: Rs.500.00   Qty: 2   SubTotal: Rs.1000.00
Product: Keyboard     Price: Rs.1200.00  Qty: 1   SubTotal: Rs.1200.00
Total Amount: Rs.57200.00

---- Test Cases ----
Added to order TEST01: TestItem
Expected Total=300.00 Actual Total=300.00

---- Bill for Order: TEST01 ----
Product: TestItem     Price: Rs.100.00   Qty: 3   SubTotal: Rs.300.00
```

```
Total Amount: Rs.300.00
```

Question 9: Student Result Analyzer using Arrays of Objects

Source Code: StudentResultAnalyzer.java

```
class Student {
    private int rollNo;
    private String name;
    private int[] marks; // 3 subjects

    public Student(int rollNo, String name, int[] marks) {
        this.rollNo = rollNo;
        this.name = name;
        this.marks = marks;
    }

    public int getRollNo() { return rollNo; }
    public String getName() { return name; }

    public int getTotal() {
        int total = 0;
        for (int m : marks) total += m;
        return total;
    }

    public double getAverage() {
        return getTotal() / (double) marks.length;
    }

    public String getGrade() {
        double avg = getAverage();
        if (avg >= 90) return "A+";
        else if (avg >= 80) return "A";
        else if (avg >= 70) return "B";
        else if (avg >= 60) return "C";
        else if (avg >= 50) return "D";
        else return "F";
    }

    public void display() {
        System.out.printf("Roll No: %-4d Name: %-10s Total: %-4d Average: %-6.2f Grade: %s%n",
                rollNo, name, getTotal(), getAverage(), getGrade());
    }
}

public class StudentResultAnalyzer {
    public static void main(String[] args) {
        Student[] students = new Student[4];
        students[0] = new Student(1, "Arun", new int[]{85, 90, 78});
        students[1] = new Student(2, "Divya", new int[]{95, 92, 98});
        students[2] = new Student(3, "Karthik", new int[]{60, 55, 70});
        students[3] = new Student(4, "Meena", new int[]{40, 35, 45});

        System.out.println("---- Student Result Analysis ----");
        for (Student s : students) s.display();

        Student topper = students[0];
        for (Student s : students) {
            if (s.getTotal() > topper.getTotal()) topper = s;
        }
        System.out.println("\nClass Topper: " + topper.getName() + " (Roll No: " + topper.getRollNo() +
") with Total: " + topper.getTotal());

        System.out.println("\n---- Test Cases ----");
        Student t1 = new Student(101, "TestA", new int[]{100, 100, 100});
        System.out.printf("TestA -> Expected Grade=A+ Actual Grade=%s%n", t1.getGrade());

        Student t2 = new Student(102, "TestF", new int[]{20, 30, 25});
        System.out.printf("TestF -> Expected Grade=F Actual Grade=%s%n", t2.getGrade());
    }
}
```

Program Output

```
---- Student Result Analysis ----
Roll No: 1    Name: Arun       Total: 253  Average: 84.33  Grade: A
Roll No: 2    Name: Divya      Total: 285  Average: 95.00  Grade: A+
Roll No: 3    Name: Karthik    Total: 185  Average: 61.67  Grade: C
Roll No: 4    Name: Meena      Total: 120  Average: 40.00  Grade: F

Class Topper: Divya (Roll No: 2) with Total: 285

---- Test Cases ----
TestA -> Expected Grade=A+ Actual Grade=A+
TestF -> Expected Grade=F Actual Grade=F
```

Question 10: Smart Home Device Controller using Hierarchical Inheritance

Source Code: SmartHomeController.java

```
class Device {
    protected String name;
    protected boolean isOn;
    protected double powerWatts;

    public Device(String name, double powerWatts) {
        this.name = name;
        this.powerWatts = powerWatts;
        this.isOn = false;
    }

    public void turnOn() {
        isOn = true;
        System.out.println(name + " turned ON.");
    }

    public void turnOff() {
        isOn = false;
        System.out.println(name + " turned OFF.");
    }

    public double getPowerConsumption() {
        return isOn ? powerWatts : 0;
    }

    public void display() {
        System.out.printf("%-15s Status: %-4s Power Consumption: %.2f W%n",
                name, isOn ? "ON" : "OFF", getPowerConsumption());
    }
}

class Light extends Device {
    public Light(String name) { super(name, 60); }
}

class Fan extends Device {
    public Fan(String name) { super(name, 75); }
}

class AirConditioner extends Device {
    public AirConditioner(String name) { super(name, 1500); }
}

public class SmartHomeController {
    public static void main(String[] args) {
        Device[] devices = new Device[3];
        devices[0] = new Light("Living Room Light");
        devices[1] = new Fan("Bedroom Fan");
        devices[2] = new AirConditioner("Hall AC");

        System.out.println("---- Smart Home Device Controller ----");
```

```
        devices[0].turnOn();
        devices[1].turnOn();
        // AC remains off

        System.out.println();
        double totalPower = 0;
        for (Device d : devices) {
            d.display();
            totalPower += d.getPowerConsumption();
        }
        System.out.printf("%nTotal Power Consumption: %.2f W%n", totalPower);

        System.out.println("\n---- Test Cases ----");
        Device testAC = new AirConditioner("TestAC");
        System.out.printf("AC off -> Expected Power=0.00 Actual=%.2f%n", testAC.getPowerConsumption());
        testAC.turnOn();
        System.out.printf("AC on -> Expected Power=1500.00 Actual=%.2f%n",
testAC.getPowerConsumption());
        testAC.turnOff();
        System.out.printf("AC off again -> Expected Power=0.00 Actual=%.2f%n",
testAC.getPowerConsumption());
    }
}
```

Program Output

```
---- Smart Home Device Controller ----
Living Room Light turned ON.
Bedroom Fan turned ON.

Living Room Light Status: ON   Power Consumption: 60.00 W
Bedroom Fan     Status: ON   Power Consumption: 75.00 W
Hall AC         Status: OFF  Power Consumption: 0.00 W

Total Power Consumption: 135.00 W

---- Test Cases ----
AC off -> Expected Power=0.00 Actual=0.00
TestAC turned ON.
AC on -> Expected Power=1500.00 Actual=1500.00
TestAC turned OFF.
AC off again -> Expected Power=0.00 Actual=0.00
```